

IUT / IFRI

Master Génie Logiciel: Théorie des langages & calculabilité

Rapport de projet formlang

De l'automate fini à la machine universelle

Binôme : KODONON M. Sunday A. · N'TCHA Kouagou Josephe

Dépôt Git : https://github.com/Sunday2000/KODONON_N-TCHA_formlang.git

Professeur : Dr. Méton ATINDEHOU

Table des matières

- 0. Résumé
- 1. Étage régulier & transducteurs (Jour 1)
- 2. Hors-contexte (Jour 2)
- 3. Arbres (Jour 3) — pivot
- 4. Calculabilité (Jour 4)
- 5. Intégration & Myhill–Nerode (Jour 5)
- 6. Difficultés rencontrées & choix de conception
- 7. Répartition du travail
- Annexe — sortie console
- Bibliographie

0. Résumé

formlang est une petite bibliothèque de théorie des langages qui traverse toute la hiérarchie de Chomsky avec **un seul objet racine, l'automate d'arbres** (`formlang.tree.TreeAutomaton`) : un mot est vu comme un arbre dégénéré (branche unaire), donc l'AFD du jour 1 est un cas particulier de l'automate d'arbres du jour 3, et la machine de Turing du jour 4 en est la généralisation « mémoire libre ». Quatre applications (`apps/shield`, `apps/morpho`, `apps/hashcons`, `apps/mtu`) **instancient** ce cœur sans jamais réécrire un automate.

Résultat final de la suite de tests :

```
$ pytest -q
.....
28 passed in 0.08s [100%]
```

1. Étage régulier & transducteurs (Jour 1)

Théorie

Q1.1: regex de L_1 . Avec $\Sigma = \{a, o, r\}$ et $L_1 = \{w \mid w \text{ contient le facteur « or »}\}$:

$L_1 = (a|o|r)^* o r (a|o|r)^*$

Q1.2: AFN $\rightarrow$ AFD $\rightarrow$ minimal. L'AFN naturel devine, à chaque `o` lu, si *celui-ci* commence l'occurrence de `or` :

état	rôle	a	o	r
q0 (init)	rien vu	q0	q0, q1	q0
q1	pari : « ce o amorce or »	—	—	q2
q2 (final)	absorbant	q2	q2	q2

Vérification en code (`NFA.to_dfa` + `DFA.minimize`, tous deux implémentés en E1.2/E1.3) sur exactement cette table :

```
AFD (sous-ensembles) avant minimisation : 4 états
δ({q0},a) = {q0}      δ({q0},o) = {q0,q1}      δ({q0},r) = {q0}
δ({q0,q1},a) = {q0}   δ({q0,q1},o) = {q0,q1}    δ({q0,q1},r) = {q0,q2}
δ({q0,q1,q2},*) = {q0,q2} ou {q0,q1,q2} (absorbant)
δ({q0,q2},*) = {q0,q2} ou {q0,q1,q2} (absorbant)
AFD minimal : 3 états
```

Les deux blocs absorbants $\{q0, q1, q2\}$ et $\{q0, q2\}$ fusionnent car ils sont tous deux acceptants et bouclent sur eux-mêmes pour les trois lettres — ils sont **indistinguables** (Myhill–Nerode, §5). L'AFD minimal obtenu est **isomorphe** à `detector_dfa()` (`apps/shield/detector.py`) : 3 états, A (rien vu), B (o vu), C (accepté, absorbant) :

```
$ (script ad hoc sur detector_dfa())
num_states(detector_dfa) = 3
δ(A,a)=A δ(A,o)=B δ(A,r)=A
δ(B,a)=A δ(B,o)=B δ(B,r)=C
δ(C,a)=C δ(C,o)=C δ(C,r)=C
minimize().num_states() = 3 # déjà minimal, la minimisation ne casse rien
```

Minimalité (justification). 3 classes de Nerode existent pour L_1 (prouvé en §5, `test_nerode_trois_classes`) $\Rightarrow$ l'AFD minimal a exactement 3 états $\Rightarrow$ `detector_dfa` est minimal.

Q1.3 — $L_1' = \{w \mid w \text{ contient un o suivi, pas forcément immédiatement, d'un r}\}$. Regex : $(a|o|r)^* o (a|o|r)^* r (a|o|r)^*$. Différence avec L_1 : L_1 demande `o` et `r` **adjacents** (facteur), L_1' seulement `o` **avant** `r` (sous-séquence) $\Rightarrow L_1 \subseteq L_1'$ strictement, témoin `oar` :

```
'oar' : contains_or=False in_L1=True # o en position 0, r en position 2 : pas adjacents
'or' : contains_or=True in_L1=True
'aoa' : contains_or=False in_L1=False
```

Miroir (one-way vs two-way). Un FST **one-way** ne peut pas réaliser $w \mapsto w^R$: sa mémoire est un **état fini** ; pour émettre w^R il faudrait avoir mémorisé tout w avant d'émettre le premier caractère de sortie (le dernier lu de w est le premier de w^R), donc distinguer au moins $|\Sigma|^n$ préfixes possibles avec un nombre d'états **indépendant de n** — impossible dès que n dépasse le nombre d'états. Un transducteur **two-way** (tête qui peut revenir en arrière) le permet trivialement : une passe pour repérer la fin, puis relecture de droite à gauche en recopiant. `reverse_twoway` (fourni) modélise cette seconde passe ; testé via `test_miroir_twoway (reverse("kcata") == "attack")`.

Pratique

E1.1. `apps/shield/detector.py` (table A/B/C) — écrit dès le commit `c0180cd` (« E1 done »).

DFA.run/accepts, en revanche, contenaient un **bug** dans ce même commit :

```
# c0180cd – premier jet, BUGUÉ
def run(self, w: str):
    accept = None
    seen = self._reachable()
    for s in seen:                # boucle sur TOUS les états atteignables...
        for l in w:
            to_check = (s,l) if accept == None else (accept,l)
            accept = self.transitions[to_check]
    return accept in self.accept
```

Défaut : la boucle externe `for s in seen` ne sert à rien dès la 2^e itération (`accept` n'est plus `None`), et surtout **elle ne part jamais garantie de `self.start`** (`seen` est un `set`, ordre non garanti) — le mot w est en pratique rejoué plusieurs fois à la suite. Corrigé (commit `6fe415a`) par la version canonique :

```
def run(self, w: str):
    state = self.start
    for l in w:
        state = self.transitions[(state, l)]
    return state in self.accept
```

E1.2. `DFA.minimize` (raffinement de Moore) — voir le récit complet en §6 (le premier jet, non commité, avait quatre défauts distincts).

E1.3. `NFA.to_dfa` — construction des sous-ensembles standard à partir de `_eps_closure/_move` (fournis) ; vérifiée ci-dessus sur l'AFN de L_1 .

E1.4. `SequentialFST.transduce` (lettre à lettre, identité si `identity_on_missing`) + `leet_fst` ($4 \rightarrow a$ $3 \rightarrow e$ $0 \rightarrow o$ $1 \rightarrow i$ $5 \rightarrow s$, un seul état q_0 final).

```
$ pytest -q tests/test_regular.py
.....
7 passed [100%]
```

2. Hors-contexte (Jour 2)

Théorie

Q2.1 — $D = \{ [^n] \mid n \geq 0 \}$ n'est pas régulier (pompage). Supposons D régulier, de longueur de pompage p . Soit $w = [^p]^p \in D$ ($|w| = 2p \geq p$). Le lemme de l'étoile décompose $w = xyz$ avec $|xy| \leq p$ et $y \neq \varepsilon$; comme xy est entièrement dans le préfixe $[^p]$, on a $y = [^k$ pour un $k \geq 1$. En pompant à $i = 0$: $xz = [^{\wedge\{p-k\}}]^p \notin D$ puisque $p - k \neq p$. Contradiction $\Rightarrow D$ n'est pas régulier. **Conséquence directe** : le `SingularityDetector` (AFD, jour 1) ne peut *jamais* vérifier une imbrication de délimiteurs — il lui faut une pile (E2.1) ou un automate d'arbres (jour 3, où le nombre de niveaux devient la *hauteur* de l'arbre, plus l'état).

Q2.2/Q2.3 — grammaire des prompts bien parenthésés et ambiguïté. `balanced_cfg()` (fournie) :

```
| S → S S | [ S ] | ( S ) | a | o | r | ε
```

Elle **est ambiguë** : $S \rightarrow S S$ n'impose aucune associativité, donc `aaa` admet (au moins) deux arbres de dérivation — plus précisément : $((a \cdot a) \cdot a)$ (associativité gauche) et $(a \cdot (a \cdot a))$ (associativité droite) sont

deux dérivations distinctes de la même chaîne `aaa` via $S \rightarrow S S$. **Désambiguïsation** — remplacer la concaténation libre par une liste :

```
| S → ε | T S          T → [ S ] | ( S ) | a | o | r
```

Vérifié en code : les deux grammaires engendrent **exactement** le même langage (mêmes ensembles de mots) jusqu'à longueur 5 :

```
max_len=3 : |G|=60   |G2|=60   identiques=True
max_len=4 : |G|=257  |G2|=257  identiques=True
max_len=5 : |G|=1160 |G2|=1160 identiques=True
```

et `G2` est non ambiguë : chaque `S` non vide a un unique découpage « premier `T`, puis reste ».

Pratique

E2.1. `DelimiterPDA.accepts` — pile `list`, empile sur ouvrant, dépile-et-compare sur fermant, ignore `a,o,r,e`, accepte ssi pile vide en fin de mot (acceptation par pile vide).

E2.2. `CFG.generate(max_len)` — **piège du jour** signalé par l'énoncé : borner uniquement la longueur des mots ne suffit pas si on énumère par dérivation gauche (formes purement non-terminales `S`, `SS`, `SSS`... jamais élaguées). Solution retenue : point fixe **par nonterminal**, sur l'ensemble (fini) des chaînes de longueur $\leq \text{max_len}$ — `lang[N]` croît de façon monotone jusqu'à stabilisation ; comme l'univers des chaînes de longueur bornée sur un alphabet fini est fini, la terminaison est garantie sans borner artificiellement le nombre de non-terminaux.

```
$ balanced_cfg().generate(max_len=4) → 257 mots, dont "" "[]" "()" "[a]" "[[]]"
et ni "[" ni "]" (délimiteurs jamais non appariés, par construction de G).
$ pytest -q tests/test_cfg_nerode.py tests/test_regular.py
.....
9 passed [100%]
```

3. Arbres (Jour 3) — pivot

Théorie

Q3.1 — déterminisme de ``morpho_automaton`/`shield_automaton``. Les deux sont construits par `add_rule(symbole, (états enfants), résultat)`, qui stocke dans un dict $\Delta[(\text{symbole}, \text{child_states})] = \text{résultat}$: une clé ne peut avoir **qu'une seule** image (propriété du type dict). Conséquence : `TreeAutomaton.run` (post-ordre) visite chaque nœud **exactement une fois** et fait une consultation $O(1)$ par nœud $\Rightarrow$ reconnaissance en $O(n)$, n = nombre de nœuds (à comparer à un parcours linéaire d'une liste de règles).

**Q3.2 — un AFD de mots échoue à valider la structure. Deux arbres de même frontière (yield) mais de structure différente, vérifiés en code* :*

```
t1 = build_word([], "penda", []) # racine seule "penda"
t2 = build_word(["pen"], "da", []) # préfixe "pen" + racine "da"
yield(t1) = 'penda'   classe = BARE
yield(t2) = 'penda'   classe = PREFIXED
```

Même frontière `"penda"`, classification différente : un AFD qui ne lit que la suite des feuilles est **aveugle** à cette différence, il lui faut l'arbre.

Q3.3 — théorème du yield. La frontière d'un mot accepté par `morpho_automaton` est exactement $p^* r s^*$ (zéro ou plusieurs préfixes, une racine, zéro ou plusieurs suffixes) — c'est le langage régulier du jour 1 retrouvé comme **image (yield)** d'un langage d'arbres régulier : le pont explicite arbres $\leftrightarrow$ mots annoncé par l'énoncé.

Q3.4 — reduplication. $\{ww \mid w \in \{a,b\}^*\}$ n'est pas régulier (pompage standard : pour $w = a^p b a^p b$, toute décomposition y pompable tombe dans le premier bloc de a , et pomper casse l'égalité des deux moitiés). Un automate d'arbres (comme un AFD) n'a qu'un **état fini** par nœud : il ne peut pas retenir une copie de taille non bornée de la racine pour la comparer à une seconde occurrence $\Rightarrow$ la reduplication n'est

pas un langage d'arbres régulier. Culy (1985) va plus loin : la reduplication du bambara réel dépasse même le hors-contexte — cf. bibliographie.

Pratique — traces et preuve d'intégration

E3.1. `TreeAutomaton.run` : post-ordre, REJECT si un enfant est REJECT ou si (symbole, `child_states`) n'est pas dans Δ ; accepts teste l'appartenance à `final`.

E3.2. `morpho_automaton` : feuilles `nil` → `NIL`, `prefix` → `PREFIX`, `suffix` → `SUFFIX`, `root` → `ROOT` ; listes `prefixes(PREFIX, {NIL, PREFIXES})` → `PREFIXES` (idem `suffixes`) ; `rest(ROOT, {NIL, SUFFIXES})` → `REST` ; `word({NIL, PREFIXES}, REST)` → `WORD` (final). Un arbre dont la racine n'est pas un `ROOT` valide (ex. un `suffix` à la place) est rejeté — testé (`test_morpho_rejet`).

E3.3. `_seq/shield_automaton` : `txt/enc` → `SAFE`, `ovr` → `OVR`, `role` → `ROLE` ; `_seq(x,y)` = `DANGER` si l'un des deux est déjà `DANGER`, sinon `_BY_SEV.get(sev(x)+sev(y), DANGER)` (somme des sévérités, sature à `DANGER` dès qu'elle dépasse `ROLE`) ; `frame/sys` → toujours `DANGER` (un faux cadre ou une fausse invite système est un signal d'attaque en soi, quel que soit son contenu — 100 % structurel, jetons abstraits `txt/enc/ovr/role/seq/frame/sys`, aucun contenu réel).

E3.4. `product(a1, a2)` : états = paires, une règle par couple de règles de même symbole/arité, finaux = paires de finaux $\Rightarrow L(a1 \times a2) = L(a1) \cap L(a2)$.

Preuve d'unité (gate anti-copier-coller). Les trois usagesinstancient `formlang.tree` sans le réécrire :

```
# apps/morpho/automaton.py
from formlang.tree import Term, TreeAutomaton
A = TreeAutomaton(final_states={"WORD"}); A.add_rule(...)

# apps/shield/decomposer.py
from formlang.tree import Term, TreeAutomaton, product
A = TreeAutomaton(final_states={DANGER}); A.add_rule(...)
return product(shield_automaton(), enc_automaton())
```

Trace d'exécution ascendante — `sys(seq(txt(), frame(role())))` (bloqué, cf. `demo_shield`) :

```
role()                → ROLE
frame(ROLE)           → DANGER   (frame toujours DANGER)
txt()                 → SAFE
seq(SAFE, DANGER)     → DANGER   (sticky : un enfant DANGER suffit)
sys(DANGER)           → DANGER   (sys toujours DANGER)
racine = DANGER ∈ {DANGER} ⇒ BLOQUÉ
$ python pipeline.py
== démo Shield (AttackDecomposer) ==
OK      seq(txt,txt)
OK      role (isolé)
BLOQUÉ  sys(role)
BLOQUÉ  seq(frame(ovr),txt)
BLOQUÉ  sys(seq(txt,frame(role)))
```

Extension P4.5 (double encodage). `enc_automaton` compte les feuilles `enc` (plafonné à 2, `final {2}`) ; `dangerous_and_double_encoded` = `product(shield_automaton(), enc_automaton())`. Un piège rencontré ici est raconté en §6 (règle `enc` manquante côté `shield_automaton`).

E3.5 — hash-consing (`apps/hashcons/store.py`). `intern` : clé canonique (`symbol`, `label`, `kids_ids`), partage si déjà vue ; `get` : reconstruction récursive exacte (round-trip, `test_round_trip_exact`). Mesures réelles :

[mini, cas du test]	total=16	uniques=9	compression=0.4375
[corpus_B-like, 40 racines × 8 suffixes]	total=2440	uniques=777	compression=0.6816
[racines seules, aucun suffixe partagé]	total=300	uniques=181	compression=0.3967

Q3.5. Le partage grimpe nettement (0.68 vs 0.40) dès qu'on ajoute des suffixes **répétés sur beaucoup de racines** — exactement le motif d'une langue **agglutinante** (turc, finnois : longues chaînes de suffixes partagées entre des milliers de racines, cf. `corpus_B/SUFFIXES_B` du projet, qui imite ce motif). Une langue **isolante** (peu d'affixation, proche du témoin « racines seules ») partage surtout les nœuds structurels (`nil`, `rest`) mais pas de longues branches suffixales $\Rightarrow$ compression plus faible. Cohérent avec Daciuk *et al.* (2000) pour le pendant « mots ».

```
$ pytest -q tests/test_tree.py tests/test_hashcons.py
.....
8 passed
```

[100%]

4. Calculabilité (Jour 4)

Théorie

Q4.1 — encodage $\langle M \rangle$. `encode/decode` (`formlang/utm.py`) linéarisent M en JSON **trié** (`sort_keys=True`) : `transitions` devient une liste de paires $[[q, a], [q', b, d]]$, `accept/reject` des listes triées. **Injectif** : deux machines de tables différentes produisent des listes différentes donc des chaînes JSON différentes (`json.dumps` est une fonction, et deux structures Python différentes ne peuvent produire la même sérialisation canonique triée). **Décodable** : `decode` reconstruit exactement le `dict`/les `set` d'origine — vérifié :

```
len(<ADD>) = 243
decode(encode(ADD)) == ADD (transitions) : True
```

Q4.2 — cycle de 'U'. `UniversalTM.run(<M>, w)` décode $\langle M \rangle$ en un objet `TuringMachine` puis délègue **directement** à `TuringMachine.run(w)` — c'est la lecture « programme enregistré » explicitement demandée par l'énoncé (*l'interpréteur n'écrit aucune boucle de ruban*) :

```
class UniversalTM:
    def run(self, encoded_machine, word, **kw):
        return decode(encoded_machine).run(word, **kw)
```

Vérifié : `U(<ADD>, "111+11")` et `ADD.run("111+11")` produisent le même ruban et la même acceptation.

```
ADD.run('111+11').tape = 11111
UniversalTM().run(<ADD>,w).tape = 11111
égalité tape : True   égalité accepted : True
```

**Q4.3 — surcoût de simulation (sourcé, honnêteté sur ce qui est vraiment mesuré). Le modèle théorique standard (Sipser, op. cit., th. 3.13) simule un multi-ruban par un mono-ruban avec un surcoût quadratique en le nombre de pas ; Hennie & Stearns (J. ACM 13(4), 1966) montrent qu'un ralentissement $O(t \log t)$ est atteignable en simulant k rubans par 2 rubans. Ces bornes concernent une machine universelle qui réécrit et fait défiler $\langle M \rangle$ sur son propre ruban à chaque pas simulé. Notre implémentation ne fait pas ça : `decode` reconstruit M en mémoire Python une fois* (coût $O(|\langle M \rangle|)$, ici 243 caractères), puis `U` appelle nativement `M.run(w)` — mesuré :*

```
steps direct (ADD.run) = 8   steps via UniversalTM = 8   (identiques)
```

Il n'y a donc **aucun** ralentissement par pas simulé dans ce code : le surcoût réel de notre `U` est le **parsing JSON une fois**, pas la borne Hennie–Stearns (qui, elle, s'applique au modèle *ruban-à-ruban* du cours, pas à notre interpréteur « méta-circulaire »). C'est le chiffre **vrai** à reporter, distinct de la borne théorique citée pour le modèle abstrait.

Q4.4 — indécidabilité. Par construction de `U` (Q4.2), `U` s'arrête sur $\langle M \rangle \# w$ **ssi** M s'arrête sur w — la question « `U` s'arrête-t-elle sur $\langle M \rangle \# w$? » est donc **exactement** `HALT`(M, w) reformulée. Or `HALT` est indécidable (Turing, 1936, argument diagonal ; Sipser op. cit., th. 4.11). Donc aucun algorithme ne décide « `U` s'arrête sur $\langle M \rangle \# w$ » en général. « Universel $\neq$ omniscient » : `U` **simule** fidèlement n'importe quelle M , mais ne peut **prédire** si cette simulation va terminer.

Pratique — lecture ligne à ligne de la table SUB

Convention. $n = 1^n$, $m = 1^m$, blanc `_`. `SUB` calcule $n - m$ tronquée à 0 en **appariant** à chaque tour un 1 de gauche avec un 1 de droite, autour d'un **pivot fixe** - (jamais déplacé) et d'un marqueur `y` (« déjà apparié ») :

transition	lecture
$(q0, 1) \rightarrow (q0, 1, R)$	on traverse n sans le modifier
$(q0, -) \rightarrow (qFindL, -, L)$	pivot trouvé, on regarde à sa gauche

transition	lecture
(qFindL,1)→(qBackL,y,R)	un 1 disponible à gauche : on le marque y
(qFindL,_)→(qZero,_,R)	plus rien à gauche : gauche épuisée, résultat 0
(qFindR,1)→(qBackR,y,L)	un 1 disponible à droite : on le marque y aussi
(qFindR,_)→(qKeep,_,L)	plus rien à droite : droite épuisée, on garde la gauche
(qKeepRestore,y)→(qKeepAfter,1,L)	on restitue l'unité de gauche marquée ce tour-ci (spéculative)
(qZero,{y,-,1})→(qZero,_,R)	tout efface (gauche, pivot, restes de droite)

Traces réelles (trace=True) :

```

2+1 = '11+1' :
t=0..2 q0 '11+1' # traverse les 2 uns
t=3 q1 '1111' # + devient 1
t=4 q1 '1111' # traverse le 1 de droite
t=5 q2 '1111_' # blanc trouvé, recule
t=6 qf '111__' # efface le dernier 1 → résultat "111" = 3 = 2+1

3-2 = '111-11' (32 pas, extrait) :
t=4 qFindL '111-11' # au pivot, regarde à gauche : un 1 → marqué y
t=6 qFindR '11y-11' # cherche à droite : un 1 → marqué y (tour 1 apparié)
... (tour 2 : encore un 1 de chaque côté, apparié)
t=17 qFindL '1yy-yy' # plus qu'un 1 à gauche, aucun à droite restant
t=24 qFindR 'yyy-yy' # droite épuisée : on garde la gauche
t=28 qKeepRestore 'yyy__' # pivot effacé
t=29 qKeepAfter 'yy1__' # l'unité spéculative de ce tour est restituée
t=32 qf '___1__' # résultat "1" = 3-2

```

Cas limites (mesurés) : $n=m$ ('11-11') → ruban ' ' (0, la gauche et la droite s'épuisent **simultanément**, branche qZero) ; $m=0$ ('111-') → ruban '111' (=n, restitution immédiate de la seule unité spéculative) ; $n=0$ ('-11') → ruban ' ' (0, qFindL trouve blanc dès le premier tour).

Table `ADD` (plus courte) : convertit + en 1 (fusion des deux blocs), traverse jusqu'au blanc, recule d'une case et efface un 1 pour compenser le 1 de trop introduit par la conversion du +.

E4.1. TuringMachine.run : ruban dict, arrêt sur état final/rejet/transition absente, directions L/R/S, trace optionnelle ((pas, état, fenêtre de ruban)).

E4.2. encode/decode/UniversalTM.run — ci-dessus.

E4.3. ADD/SUB (tables ci-dessus) + Calculatrice : addition/soustraction **exécutent la vraie MT** ; multiplication = additions répétées ; division = soustractions répétées (comptées) ; chainer compose. Tests exhaustifs :

```

for n,m in range(7)×range(7): addition==n+m, soustraction==max(0,n-m),
                                multiplication==n*m, division==divmod(n,m)
→ 49 combinaisons, toutes vérifiées (test_calculatrice_exhaustif : PASSED)
chainer(2, [( "+",1), ("*",2), ("-",1), ("/",2)]) == 2 (2+1=3, ×2=6, -1=5, ÷2=(2,1)→2)
division(3, 0) → ZeroDivisionError (levée)

```

E4.4. UniversalInterpreter.run(M, w) = self._U.run(encode(M), w) ; addition_via_utm(3,2) == 5, soustraction_via_utm(5,2) == 3 — la calculatrice **exécutée par `U`**, pas par un appel direct à M.run.

```

$ pytest -q tests/test_turing.py tests/test_calc.py
.....
8 passed [100%]

```

5. Intégration & Myhill–Nerode (Jour 5)

Intégration bout-en-bout

pipeline.py ne réimplémente rien ; il enchaîne les applications :


```
$ python pipeline.py --word 4or
      brut : 4or
      normalisé(FST) : aor
      facteur_or(AFD) : True
      délimiteurs_ok(PDA) : True

$ python pipeline.py --morpho mufafak
{'mot': 'mufafak', 'classe(BUTA)': 'PREFIXED'}
```

`analyze_word` : `leet_normalize` (FST) → `contains_or` (AFD) → `well_parenthesized` (PDA), trois étages de la hiérarchie enchaînés sur la même entrée. `analyze_morpho` : `discover` (heuristique d'alternance, Harris 1955/Goldsmith 2001) → `segment_to_tree` → `classify` (BUTA).

Myhill–Nerode

Q5.1/Q5.2. $u \approx_L v \text{ssi } \forall w, uw \in L \Leftrightarrow vw \in L$ — congruence à droite, indice fini $\Leftrightarrow L$ régulier, et l'indice = le nombre d'états de l'AFD minimal. `nerode_classes` approxime ce critère par une batterie **finie** de suffixes témoins (comparer sur *tous* les suffixes serait infini) :

```
words = ["", "a", "o", "ao", "or", "aor", "oo", "oa", "ror"]
suffixes = ["", "r", "or", "a"]
→ 3 classes :
  classe 0 : ['', 'a', 'oa']      # "rien de spécial"
  classe 1 : ['ao', 'o', 'oo']    # "je viens de voir un o"
  classe 2 : ['aor', 'or', 'ror'] # "or" déjà vu, c'est joué
equivalent('o', 'ao') = True   equivalent('o', 'a') = False
```

Ces **3 classes** sont exactement les 3 états A, B, C de l'AFD minimal du jour 1 (§1) — la minimisation de Moore n'est rien d'autre que le calcul de ces classes : fusionner deux états, c'est constater que les mots qui y mènent sont Nerode-équivalents.

```
$ pytest -q tests/test_pipeline.py tests/test_cfg_nerode.py
..... [100%]
5 passed
```

6. Difficultés rencontrées & choix de conception

1) DFA.run — le bug de « E1 done »

Décrit en §1 : boucler sur `_reachable()` au lieu de partir de `self.start` fonctionne « par accident » sur des cas triviaux mais rejoue le mot plusieurs fois dès que l'automate a plus d'un état atteignable. Diagnostiqué en reconstruisant à la main l'exécution pas à pas (`accept` ne redevient jamais `None`, donc l'état de départ réel dépend de l'ordre arbitraire d'itération d'un `set`). Corrigé en suivant strictement la définition : `state = self.start`, puis `state = δ(state, lettre)`.

2) DFA.minimize/moore — quatre bugs dans le premier jet (non commité)

Avant 6fe415a, un premier brouillon existait dans l'arbre de travail :

```
def moore(self, *packets):
    signatures = []
    diff_packets = packets[1:]
    f_pack = copy.deepcopy(packets[0])
    ...
    for state in packets[0]:
        ...
        if signature not in signatures:
            signatures.append(signature)
            diff_packets = ({state},) + diff_packets
            f_pack = f_pack - {state}
        ...
    if len(signatures) > 1:
        return self.moore(f_pack, *diff_packets)
```

Défauts identifiés :

- **ne traite jamais que ``packets[0]``** : un bloc qui a besoin d'être scindé n'est jamais réexaminé s'il n'est pas en première position (typiquement le bloc des acceptants) ;
- **regroupement par signature faux** : seul le *premier* état de chaque signature *nouvelle* est isolé, les suivants restent lumpés ensemble même s'ils ont des signatures différentes. Contre-exemple construit à la main : 4 états A, B, C, D de signatures X, Y, X, Y — l'algorithme les scinde en 4 singletons au lieu de {A, C} et {B, D} ;
- **encodage de signature ambigu** (concaténation `str(i)` sans séparateur, collision possible dès ≥ 10 blocs) ;
- **critère d'arrêt local** (`len(signatures) > 1`) au lieu d'un point fixe sur toute la partition ;
- ruban non complété (`self.transitions[...]` direct au lieu de `self._completed()`) $\Rightarrow$ `KeyError` sur un AFD partiel.

Réécrit avec un regroupement par **dictionnaire signature** $\rightarrow$ **ensemble** (pas de mélange possible), sur **tous** les blocs à chaque passage, arrêt sur `len(new_packets) == len(packets)` (point fixe global), et `self._completed()` en tête de `moore` et de `minimize`. Vérifié sur un AFD à 5 états conçu pour forcer une vraie fusion non triviale :

```
avant minimisation : 5 états (q0..q4)
après minimisation : 3 états - {q0}, {q1,q2}, {q3,q4}
équivalence d.accepts(w) == m.accepts(w) vérifiée sur 9 mots témoins
```

3) SUB — la mémoire non bornée n'a pas de « bord »

Premier réflexe : faire *avancer* le pivot - vers la gauche à chaque tour (« manger » le 1 adjacent). Ça casse dès qu'on a besoin de revenir chercher le prochain 1 à droite : après avoir effacé l'ancien pivot, la cellule devient **blanche**, indiscernable d'une vraie fin de ruban — le ruban dict est **bi-infini**, il n'y a pas de « case 0 » détectable par un simple « scanner jusqu'au blanc ». Solution : garder le pivot **fixe** et marquer les unités déjà appariées avec un symbole dédié `y` (jamais confondu avec un blanc « vierge ») ; et parce que la marque de gauche est posée **avant** de savoir si un partenaire existe à droite, prévoir un état `qKeepRestore` qui rend l'unité si le côté droit s'avère vide — sinon `3-0` renvoyait `2` au lieu de `3`. Les cas `n=m`, `m=0`, `n=0` sont vérifiés en §4.

4) enc_automaton / product — règle manquante côté shield_automaton

`product` ne crée une règle pour un symbole que si **les deux** automates ont une règle pour ce symbole/arité (`test_shield_double_encodage_P45` échouait : toute feuille `enc` faisait rejeter le produit, faute de règle (`"enc", ()`) dans `shield_automaton`). Corrigé en ajoutant `shield_automaton().add_rule("enc", (), SAFE)` — un encodage seul n'est pas dangereux, seule sa **répétition** (comptée par `enc_automaton`) l'est.

5) CFG.generate — piège annoncé par l'énoncé, confirmé en pratique

Voir §2 : borner seulement la longueur des *terminaux* laisse passer des formes purement non-terminales infinies. Le point fixe par nonterminal (§2) contourne le piège sans avoir à borner artificiellement un nombre de non-terminaux.

Commits vs granularité réelle

L'historique Git ne compte que 3 commits (`da4f608` squelette, `c0180cd` premier jet E1.1, `6fe415a` lot E1.2 $\rightarrow$ E3.2) ; le travail E3.3 $\rightarrow$ E5 (`decomposer.py` P4.5, `tree.py::product`, `turing.py`, `utm.py`, `apps/mtu/*`, `myhill_nerode.py`, `hashcons/store.py`, `pipeline.py` — 179 insertions / 59 suppressions sur 10 fichiers) était encore **non commité** au moment de la rédaction de ce rapport.

⚠ À committer avant le rendu (cf. livrable Git, §8 de l'énoncé).

7. Répartition du travail

⚠ À compléter par le binôme avant le rendu.

Jour / étage	Membre
1 — Régulier & transducteurs	...
2 — Hors-contexte	...
3 — Arbres & hash-consing	...
4 — Calculabilité	...
5 — Intégration & Nerode	...
Rapport	...

Annexe — sortie console

pytest -q (complet)

```
$ pytest -q
.....
28 passed in 0.08s [100%]
```

python pipeline.py (3 modes)

```
$ python pipeline.py
== démo Shield (AttackDecomposer) ==
OK      seq(txt,txt)
OK      role (isolé)
BLOQUÉ  sys(role)
BLOQUÉ  seq(frame(ovr),txt)
BLOQUÉ  sys(seq(txt,frame(role)))

$ python pipeline.py --word 4or
      brut : 4or
  normalisé(FST) : aor
   facteur_or(AFD) : True
délimateurs_ok(PDA) : True

$ python pipeline.py --morpho mufafak
{'mot': 'mufafak', 'classe(BUTA)': 'PREFIXED'}
```

Bibliographie

Références vérifiées, reprises et consolidées à partir des cinq fiches du projet. Les références marquées (*libre*) sont disponibles gratuitement en ligne.

Automates finis & langages formels

- M. Sipser, *Introduction to the Theory of Computation*, 3^e éd., Cengage, 2012.
- J. Hopcroft, R. Motwani, J. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3^e éd., Pearson, 2006.
- J. Hopcroft, J. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Addison-Wesley, 1979 — ch. 3.

Automates d'arbres

- H. Comon, M. Dauchet, R. Gilleron, C. Löding, F. Jacquemard, D. Lugiez, S. Tison, M. Tommasi, *Tree Automata Techniques and Applications (TATA)*, 2007. (*libre* : tata.gforge.inria.fr)

- F. Gécseg, M. Steinby, *Tree Automata*, Akadémiai Kiadó, 1984 ; réédition (*libre* : *arXiv:1509.06233*, 2015).
- J. W. Thatcher, J. B. Wright, « Generalized finite automata theory with an application to a decision problem of second-order logic », *Mathematical Systems Theory* 2(1):57–81, 1968.
- J. Doner, « Tree acceptors and some of their applications », *Journal of Computer and System Sciences* 4(5):406–451, 1970.

Applications des automates d'arbres

- M. Murata, D. Lee, M. Mani, K. Kawaguchi, « Taxonomy of XML schema languages using formal language theory », *ACM Trans. on Internet Technology* 5(4):660–704, 2005.
- A. V. Aho, M. Ganapathi, S. W. K. Tjiang, « Code generation using tree matching and dynamic programming », *ACM TOPLAS* 11(4):491–516, 1989.
- A. Bouajjani, P. Habermehl, A. Rogalewicz, T. Vojnar, « Abstract Regular Tree Model Checking », *ENTCS* 149(1):37–48, 2006.

Morphologie computationnelle

- K. Koskenniemi, *Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production*, Université d'Helsinki, 1983.
- K. R. Beesley, L. Karttunen, *Finite State Morphology*, CSLI Publications, 2003.
- Z. S. Harris, « From phoneme to morpheme », *Language* 31(2):190–222, 1955.
- J. Goldsmith, « Unsupervised learning of the morphology of a natural language », *Computational Linguistics* 27(2):153–198, 2001.
- E. Roche, Y. Schabes (dir.), *Finite-State Language Processing*, MIT Press, 1997.

Réduplication & limites

- C. Culy, « The complexity of the vocabulary of Bambara », *Linguistics and Philosophy* 8:345–351, 1985.
- B. Roark, R. Sproat, *Computational Approaches to Morphology and Syntax*, Oxford University Press, 2007.

Hash-consing & partage de structure

- J. Daciuk, S. Mihov, B. Watson, R. Watson, « Incremental construction of minimal acyclic finite-state automata », *Computational Linguistics* 26(1), 2000.
- J.-C. Filliâtre, S. Conchon, « Type-safe modular hash-consing », *ML Workshop*, 2006.

Calculabilité & machine universelle

- A. M. Turing, « On Computable Numbers, with an Application to the Entscheidungsproblem », *Proc. London Math. Soc.*, s2-42:230–265, 1936-37.
- M. Sipser, *op. cit.* — machine universelle, encodage, indécidabilité, réductions.
- F. C. Hennie, R. E. Stearns, « Two-Tape Simulation of Multitape Turing Machines », *Journal of the ACM* 13(4):533–546, 1966.
- S. Arora, B. Barak, *Computational Complexity: A Modern Approach*, Cambridge University Press, 2009.